UNIVERSITY AT ALBANY
State University of New York

College of Engineering and Applied Sciences
Computer Science Department

A Case of Restricted Negation in Access-Control Policies

(Master's Project)

by

Haroun Moussa Hamza

Under the Guidance of
Professor

Dr. Paliath Narendran

(pnarendran@albany.edu)

Abstract

This project studies whether Boolean formulas can be represented in the form $\phi_1 \wedge \neg\phi_2$, where ϕ_1 is a Horn-DNF and ϕ_2 is an all-positive DNF. The motivation of the problem comes from the representability problem in access-control policies with `PERMIT` and `DENY` rules.

Acknowledgements

I would like to express my sincere gratitude to Professor Paliath Narendran for his guidance, support, and valuable feedback throughout this project. I am also thankful to Josue Ruiz and the other team members involved in this work for their time, discussion, and collaboration. Finally, I would like to thank my family for their encouragement and support during my studies.

AI Assistance Statement

AI tools, including **ChatGPT** and **Gemini**, were used in a limited supporting role during this project. They were mainly used to help reorganize code into a single-file format, assist with LaTeX formatting, improve paraphrasing, and support documentation. The core research direction, final decisions, testing, and verification were carried out by the author.

1 Introduction

Boolean formulas are widely used to represent logical rules, constraints, and decision conditions in computer science. They arise in areas such as automated reasoning, knowledge representation, formal verification, access control, and rule-based systems. In many settings, the efficiency of reasoning depends not only on the meaning of a formula, but also on the structural form in which it is expressed. For this reason, identifying special classes of Boolean formulas that admit simpler representations or more efficient reasoning procedures is an important problem.

This project studies whether Boolean formulas can be represented in the form $\phi \equiv \phi_1 \wedge \neg\phi_2$, where ϕ_1 is a Horn-DNF and ϕ_2 is an all-positive DNF. More specifically, given a Boolean formula ϕ in disjunctive normal form (DNF), the goal is to determine whether there exist formulas ϕ_1 and ϕ_2 of these restricted types such that the above equivalence holds.

The proposed approach is based on a resolution-driven transformation pipeline. Starting from the input formula ϕ, we first convert it into an equivalent conjunctive normal form (CNF), denoted by ϕ_c. We then compute the resolution closure $\widehat{\phi_c}$ and partition the resulting clauses into two parts according to polarity. The first part, denoted by ϕ_{OP}, contains all clauses with at least one positive literal. The second part, denoted by ϕ_N, contains all-negative clauses only. Next, we define $\psi_2 := \neg\phi_N$, and convert ϕ_{OP} into DNF, denoted by ψ_1.

The main question is whether this process yields a representation equivalent to the original formula and, in particular, whether the DNF obtained from the ϕ_{OP} side can be reduced to a Horn-DNF. To study this, the project computes the consensus closure of $\psi_1 \vee \psi_2$ and examines whether all non-Horn terms are redundant. Thus, the problem is not only one of syntactic transformation, but also one of identifying the structural conditions under which the decomposition succeeds.

The motivation for studying this form is that it separates a Boolean formula into two components with clear restrictions: one Horn-DNF part and one negated all-positive-DNF part. Such a decomposition may provide a more transparent way to analyze formulas and may support simplification and reasoning more effectively than an unrestricted representation.

The contribution of this project is therefore a characterization study of this decomposition process. It investigates how resolution closure, clause partitioning, DNF conversion, and consensus interact in determining whether a formula admits a representation of the form $\phi \equiv \phi_1 \wedge \neg\phi_2$, where ϕ_1 is Horn-DNF and ϕ_2 is all-positive DNF.

The remainder of this project is organized as follows. First, we introduce the basic definitions and notation used throughout the paper. Next, we describe the resolution-based transformation pipeline in detail. We then develop the main structural results needed to analyze the decomposition and study the conditions under which the resulting representation is correct.

2 Preliminaries and Notation

In this section, we introduce the notation and basic concepts used throughout the project. We work in standard propositional logic over a finite set of Boolean variables

$$X = \{x_1, x_2, \ldots, x_n\}.$$

2.1 Basic Definitions

A *literal* is either a variable x_i (a positive literal) or its negation $\neg x_i$ (a negative literal). A *clause* is a disjunction of literals, and a *term* is a conjunction of literals. A formula is in *conjunctive normal form* (CNF) if it is a conjunction of clauses, and it is in *disjunctive normal form* (DNF) if it is a disjunction of terms.

We classify clauses according to the polarity of their literals:

- a **positive clause** contains only positive literals,
- a **negative clause** contains only negative literals,
- a **mixed clause** contains both positive and negative literals.

2.2 Horn and Dual-Horn Structures

Definition 1 (Horn Clause and Horn CNF)**.** *A clause is called **Horn** if it contains at most one positive literal. A CNF formula is a **Horn CNF** if each of its clauses is Horn.*

Definition 2 (Horn DNF)**.** *A DNF formula is called a **Horn DNF** if each of its terms contains at most one negative literal.*

Horn formulas are important because they admit useful closure properties and often support efficient reasoning procedures. In this project, Horn DNF appears on the ϕ_1 side of the target representation.

2.3 Boolean Functions and True Vectors

A Boolean formula defines a Boolean function

$$f : \{0,1\}^n \to \{0,1\}.$$

We write $T(f)$ for the set of true vectors (models) of f, and $F(f)$ for the set of false vectors of f.

Definition 3 (OR-Closure)**.** *A set $S \subseteq \{0,1\}^n$ is said to be **closed under bitwise OR** if for every $u, v \in S$, the vector $u \vee v$ also belongs to S.*

Definition 4 (Horn Representability)**.** *A Boolean function f is said to be **Horn representable** if it can be represented by a Horn DNF.*

A key fact used later is that a Boolean function f is Horn representable if and only if its set of false points/vectors $F(f)$ is closed under bitwise **and** ($\wedge$).

2.4 Resolution and Resolution Closure

Resolution is an inference rule on clauses. If

$$C_1 = (A \vee x) \quad \text{and} \quad C_2 = (B \vee \neg x),$$

then the *resolvent* of C_1 and C_2 on the variable x is

$$(A \vee B).$$

Definition 5 (Subsumption)**.** *Let C_1 and C_2 be two clauses. We say that C_1* subsumes *C_2 if every literal in C_1 also appears in C_2. Equivalently, $C_1 \subseteq C_2$ as sets of literals. In this case, C_2 is redundant.*

Definition 6 (Resolution Closure)**.** *Let ϕ_c be a CNF formula. The **resolution closure** of ϕ_c, denoted by $\widehat{\phi_c}$, is the set obtained by repeatedly applying the resolution and subsumption rules to clauses of ϕ_c until no new clauses can be added.*

In this project, resolution closure is used to obtain a saturated clause set before performing the polarity-based partition. Its role is to expose logically implied clauses ("implicates") that may be hidden in the initial CNF presentation.

2.5 Consensus and Consensus Closure

Since the final step of the proposed approach uses consensus closure, we also recall that notion here.

Definition 7 (Consensus of Two Terms)**.** *Let t_1 and t_2 be two terms in a DNF formula. If there is exactly one variable x such that x appears positively in one term and negatively in the other, and all other literals are compatible, then the **consensus** of t_1 and t_2 is the term obtained by removing x and $\neg x$ and taking the conjunction of the remaining literals.*

Definition 8 (Consensus Closure)**.** *The **consensus closure** of a DNF formula is the set obtained by repeatedly adding all consensus terms that can be formed from pairs of its terms until no new terms can be added.*

In this project, consensus closure is used in the final stage of the method when analyzing $\psi_1 \vee \psi_2$. The goal is to check whether all non-Horn terms are redundant, and therefore whether the resulting formula is equivalent to a Horn-DNF.

3 The Task

Our goal is to design a method to check whether a DNF formula can be represented as

$$\psi_1 \wedge \neg\psi_2$$

where ψ_1 is a Horn-DNF and ψ_2 is a positive DNF. In other words, the computational task is:

Instance: A DNF formula ϕ.

Question: Are there formulae ψ_1 and ψ_2 such that ψ_1 is a Horn-DNF, ψ_2 is an all-positive DNF and $\phi \cong \psi_1 \wedge \neg\psi_2$?

We refer to $(\psi_1,\ \psi_2)$ as a *representation* for ϕ.

Example 1. $\phi = x_1\overline{x_2} \vee \overline{x_2x_3}$

We can take $\psi_1 = x_1 \vee \overline{x_3}$*, and* $\psi_2 = x_2$.

Example 2. $\phi = x_1 \vee \overline{x_2x_3}$

This has no solution. First of all note that setting x_1, x_2 and x_3 to be True results in a satisfying assignment for ϕ. (i.e., $111 \in T(\phi)$). So if ϕ has a representation $(\psi_1,\ \psi_2)$, then ψ_2 has to be False since otherwise $x_1, x_2, x_3 :=$ True will not be a satisfying assignment.

4 The Resolution-Based Transformation Pipeline

In this section, we describe the method used to solve the following question: given a DNF formula ϕ, can it be represented in the form

$$\phi \cong \phi_1 \wedge \neg\phi_2,$$

where ϕ_1 is a Horn-DNF and ϕ_2 is an all-positive DNF?

Formally,

Instance: A DNF formula ϕ.

Question: Do there exist formulas ϕ_1 and ϕ_2 such that ϕ_1 is a Horn-DNF, ϕ_2 is an all-positive DNF, and

$$\phi \cong \phi_1 \wedge \neg\phi_2?$$

The proposed approach is based on resolution closure, polarity-based partition, and a final consensus check.

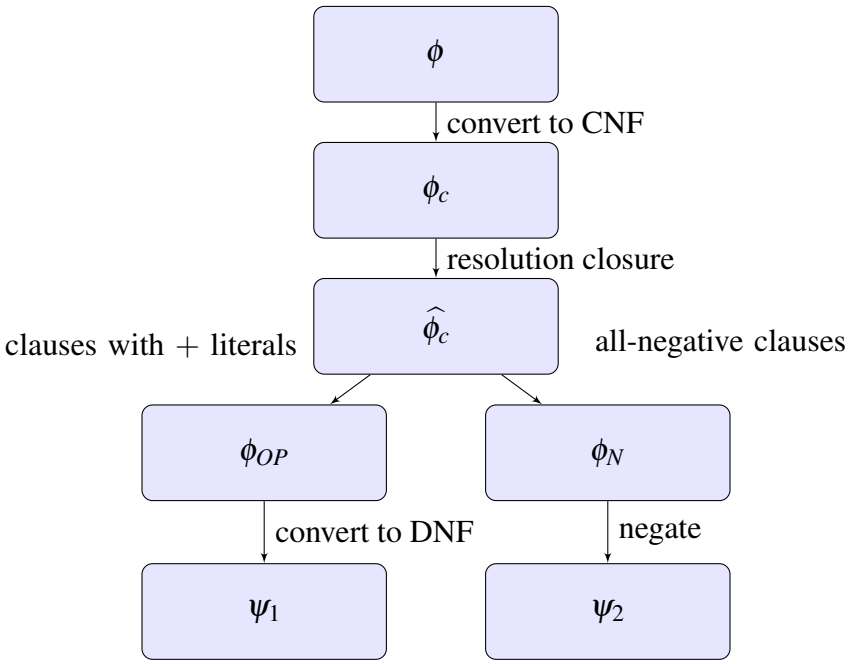


Figure 1: Resolution-based transformation pipeline.

Starting from the input formula ϕ, the method proceeds as follows:

1. Convert ϕ into an equivalent CNF, denoted by ϕ_c.
2. Compute the resolution closure of ϕ_c, denoted by $\widehat{\phi_c}$.

3. Split $\widehat{\phi_c}$ into two parts:

$$\widehat{\phi_c} = \phi_{OP} \wedge \phi_N,$$

where ϕ_N consists of all-negative clauses, and ϕ_{OP} consists of clauses with at least one positive literal, i.e.,

$$\phi_{OP} = \left\{ C \in \widehat{\phi_c} \mid C \text{ contains at least one positive literal} \right\},$$
$$\phi_N = \left\{ C \in \widehat{\phi_c} \mid C \text{ contains no positive literals} \right\}.$$

4. Define

$$\psi_2 := \neg\phi_N.$$

5. Convert ϕ_{OP} into DNF, and call the result ψ_1.

6. If ψ_1 is already a Horn-DNF, we are done; otherwise compute the consensus closure of $\psi_1 \vee \psi_2$, and check whether it is equivalent to a Horn-DNF, that is, whether all non-Horn terms are redundant.

5 Future Work

The present work includes the implementation of the proposed method and testing on a number of examples. The main remaining direction is therefore theoretical rather than experimental. In particular, future work should establish a formal correctness proof for the method and identify precise conditions under which the resolution-based split together with the consensus step yields a valid representation.

A second direction is to develop this method further into a complete decision procedure for the problem. If such an algorithm can be established, it would provide a more systematic way to decide whether a given DNF formula admits a representation of the required form.

References

[1] Yves Crama and Peter L. Hammer. *Boolean Functions - Theory, Algorithms, and Applications*, volume 142 of *Encyclopedia of mathematics and its applications*. Cambridge University Press, 2011.

[2] Hans de Nivelle. Propositional resolution. Lecture notes.

[3] Thomas Eiter, Toshihide Ibaraki, and Kazuhisa Makino. Bidual horn functions and extensions. *Discrete Applied Mathematics*, 96–97:55–88, 1999. doi:10.1016/S0166-218X(99)00112-4.

[4] Padmavathi Iyer, Amirreza Masoumzadeh, and Paliath Narendran. On the expressive power of negated conditions and negative authorizations in access control models. *Computers & Security*, 116:102586, 2022.

[5] Alex Kean and George K. Tsiknis. An incremental method for generating prime implicants/impicates. *Journal of Symbolic Computation*, 9(2):185–206, 1990. doi:10.1016/S0747-7171(08)80029-6.

[6] Roni Khardon. Translating between horn representations and their characteristic models. Technical Report TR-03-95, Harvard University, 1995.

[7] Amirreza Masoumzadeh, Paliath Narendran, and Padmavathi Iyer. Towards a theory for semantics and expressiveness analysis of rule-based access control models. In *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*, SACMAT '21, pages 33–43. Association for Computing Machinery, 2021. doi:10.1145/3450569.3463569.

[8] Robert McNaughton. Unate truth functions. *IRE Transactions on Electronic Computers*, EC-10(1):1–6, 1961. doi:10.1109/TEC.1961.5219145.

[9] Paliath Narendran. Sat and resolution notes. Class notes.

[10] Josué A. Ruiz. *Studies on Convexity of DNF Formulae*. PhD thesis, University at Albany, State University of New York, 2025.

[11] Josué A. Ruiz, Paliath Narendran, Amir Masoumzadeh, and Padmavathi Iyer. Converting rule-based access control policies: From complemented conditions to deny rules. In *Proceedings of the 29th ACM Symposium on Access Control Models and Technologies, SACMAT 2024, San Antonio, TX, USA, May 15-17, 2024*. ACM, 2024. doi:10.1145/3649158.3657040.

Appendix A: The code

This appendix contains the Python implementation.

The code is commented so that each major step of the pipeline is easier to follow. In particular, the implementation includes the conversion from DNF to CNF, resolution closure, polarity-based partition, consensus closure, and the final Horn-DNF equivalence test.
In addition to the implementation itself, the code was also checked using unit tests for the main helper functions and for representative success and failure examples.

How to Run the Code

The Python implementation can be executed from the command line. The solver supports menu mode, built-in examples, direct DNF input, interactive input mode, full output, and JSON output.

Run with menu mode:

```
python3 Boolean_solver.py
```

Then choose one of the following options when prompted:

```
1  - Run built-in example 1
2  - Run built-in example 2
3  - Enter your own DNF
```

Run a built-in example directly:

```
python3 Boolean_solver.py --example 1
python3 Boolean_solver.py --example 2
```

Run with a custom DNF input:

```
python3 Boolean_solver.py --dnf "[[1,-2,-3],[4]]"
```

Run in interactive mode:

```
python3 Boolean_solver.py --interactive
```

Run the unit tests:

```
python3 -m unittest test_Boolean_solver.py
```

GitHub Repository: `https://github.com/hamzharo/restricted-negation-solver-SCSI661-S26.git`

The full implementation, test file, and README documentation are available in the GitHub repository above.

```python
from __future__ import annotations

# This program tests whether an input Boolean formula in DNF
# can be written in the form:
#     phi    phi1     phi2
# where:
#     phi1 is a Horn-DNF
#     phi2 is an all-positive DNF

"""
Solver for Boolean formulas.

Goal:
    Given a DNF formula phi, test whether it fits the pattern

        phi    phi1     phi2

    where:
        - phi1 is a Horn-DNF
        - phi2 is an all-positive DNF

Main pipeline:
    1. Convert input DNF to CNF
    2. Compute resolution closure of the CNF
    3. Split the closure into:
         - phi_N  : all-negative clauses
         - phi_OP : all remaining clauses
    4. Build:
         - psi2 = phi_N   (all-positive DNF)
         - psi1 = DNF(phi_OP)
    5. Compute consensus closure of psi1  psi2
    6. Keep only Horn terms and test equivalence
"""

# argparse is used to read command-line options such as:
# --example, --interactive, --dnf, --json, --full
import argparse

# ast is used to safely read Python-style text input such as:
# [[1,2],[-1,3]]
import ast

# json is used when the user wants JSON output instead of formatted terminal output
import json

# shutil is used here to detect the terminal width
# so that separator lines can fit the screen better
import shutil

# sys is used here to check whether the output is going to a real terminal
# this helps decide whether color should be used
import sys

# dataclass is used to store the final solver result in a clean structured way
# asdict converts the dataclass result into a dictionary
from dataclasses import dataclass, asdict

# These typing imports make the code easier to understand.
# They tell us what kind of data each function expects or returns.
from typing import Dict, Iterable, List, Optional, Sequence, Set, Tuple

# SymPy is used for symbolic Boolean logic.
# And, Or, Not, Symbol are used to build Boolean expressions.
# true and false represent Boolean constants.
from sympy import And, Or, Not, Symbol, true, false

# to_cnf converts formulas to CNF.
# to_dnf converts formulas to DNF.
from sympy.logic.boolalg import to_cnf, to_dnf
```

```
# satisfiable is used to test whether two Boolean formulas are equivalent.
# In this code, equivalence is checked by asking whether XOR is unsatisfiable.
from sympy.logic.inference import satisfiable

# A Literal is represented by an integer.
# Example:
#   3  means x3
#  -3  means x3
Literal = int

# A Term is a conjunction (AND) of literals.
# Example:
#   [1, -2, 4] means x1   x2     x4
Term = List[Literal]

# A Clause is a disjunction (OR) of literals.
# Example:
#   [1, -2, 4] means x1   x2     x4
Clause = List[Literal]

# DNF is a list of terms.
# Example:
#   [[1,2],[-1,3]] means (x1  x2)    ( x1     x3)
DNF = List[Term]

# CNF is a list of clauses.
# Example:
#   [[1,2],[-1,3]] means (x1  x2)    ( x1     x3)
CNF = List[Clause]


# ============================================================
# Styling helpers for terminal output
# ============================================================

# USE_COLOR is True when the program is running in a real terminal.
# If the output is redirected to a file or another environment,
# color may be turned off.
USE_COLOR = sys.stdout.isatty()


# This class stores ANSI color codes for nicer terminal output.
# If USE_COLOR is False, all of these become empty strings.
# That way the same print code still works without colors.
class C:
    RESET = "\033[0m" if USE_COLOR else ""
    BOLD = "\033[1m" if USE_COLOR else ""
    DIM = "\033[2m" if USE_COLOR else ""
    GREEN = "\033[32m" if USE_COLOR else ""
    RED = "\033[31m" if USE_COLOR else ""
    YELLOW = "\033[33m" if USE_COLOR else ""
    CYAN = "\033[36m" if USE_COLOR else ""
    MAGENTA = "\033[35m" if USE_COLOR else ""
    BLUE = "\033[34m" if USE_COLOR else ""


# This function checks the terminal width.
# The fallback size is (90, 24) if the width cannot be detected.
# The width is used to print clean separator lines.
def term_width() -> int:
    return shutil.get_terminal_size((90, 24)).columns


# This function builds a horizontal line using the given character.
# Example:
#   hr() gives a long line of
#   hr("  ") gives a long line of
# The line length is limited by the terminal width.
def hr(ch: str = " ") -> str:
    return ch * min(term_width(), 90)
```

```python
# This function prints a main title banner.
# It is used for larger headings such as:
# SOLVER
def banner(title: str) -> None:
    print(f"\n{C.CYAN}{C.BOLD}{title}{C.RESET}")
    print(f"{C.CYAN}{hr()}{C.RESET}")


# This function prints a smaller section title.
# It is used for parts such as:
#   Input
#   Counts
#   Candidates
#   Final result
def section(title: str) -> None:
    print(f"\n{C.BLUE}{C.BOLD}{title}{C.RESET}")
    print(f"{C.BLUE}{hr()}{C.RESET}")


# This function converts a True/False value into a colored word.
# True  -> YES in green
# False -> NO in red
def status_word(ok: bool) -> str:
    return f"{C.GREEN}{C.BOLD}YES{C.RESET}" if ok else f"{C.RED}{C.BOLD}NO{C.RESET}"


# This function turns a value into a short one-line string.
# It removes spaces and shortens very long output with "...".
# This helps keep terminal output compact and readable.
def compact(obj, max_len: int = 110) -> str:
    s = str(obj).replace(" ", "")
    if len(s) <= max_len:
        return s
    return s[:max_len] + " ..."


# This function prints a label and a compact value on one line.
# It is used for items such as:
#   Input DNF
#   psi1
#   psi2
def show_line(label: str, value, max_len: int = 110) -> None:
    print(f"{C.BOLD}{label:<28}{C.RESET} {compact(value, max_len)}")


# This function prints a label and an integer count.
# It is used in the Counts section.
def show_count(label: str, value: int) -> None:
    print(f"{C.BOLD}{label:<28}{C.RESET} {value}")


# ============================================================
# Basic helpers
# ============================================================

# This function puts literals into a standard canonical order.
# It also removes duplicates by converting to a set first.
# This helps compare clauses and terms consistently.
# Example:
#   [3, -1, 3, 2] may become (-1, 2, 3)
def canon_lits(lits: Iterable[Literal]) -> Tuple[Literal, ...]:
    """Canonical tuple form for a term/clause."""
    return tuple(sorted(set(lits), key=lambda x: (abs(x), x < 0, x)))


# This function negates one literal.
# Example:
#   3  -> -3
#  -2 -> 2
```

```python
def negate_lit(x: Literal) -> Literal:
    return -x


# This function negates every literal in a clause.
# In this code, it is used when building a term from a clause.
# Example:
#   [1, -3, 4] -> [-1, 3, -4]
def negate_clause(cls: Clause) -> Term:
    """Negation of a clause gives a term."""
    return [negate_lit(x) for x in cls]


# This function checks whether every literal is positive.
# Example:
#   [1,2,4]    -> True
#   [1,-2,4]   -> False
def all_positive(lits: Sequence[Literal]) -> bool:
    return all(x > 0 for x in lits)


# This function checks whether every literal is negative.
# Example:
#   [-1,-2,-4] -> True
#   [1,-2,-4]  -> False
def all_negative(lits: Sequence[Literal]) -> bool:
    return all(x < 0 for x in lits)


# A Horn clause is a clause with at most one positive literal.
# This function checks that condition.
def is_horn_clause(cls: Sequence[Literal]) -> bool:
    """Horn clause = at most one positive literal."""
    return sum(1 for x in cls if x > 0) <= 1


# A Horn-DNF term is a term with at most one negative literal.
# This function checks that condition.
def is_horn_term(term: Sequence[Literal]) -> bool:
    """Horn DNF term = at most one negative literal."""
    return sum(1 for x in term if x < 0) <= 1


# This improved version safely handles empty formulas.
# It collects all variable indices first.
# If there are no literals at all, it returns 0.
# Otherwise, it returns the largest variable number used.
def highest_var(formula: Sequence[Sequence[Literal]]) -> int:
    lits = [abs(x) for part in formula for x in part]
    if not lits:
        return 0
    return max(lits)


# ---------------------------------------------------------------------

# This function checks whether a term or clause is consistent.
# It rejects expressions that contain both x and -x.
# Example:
#   [1, -1, 3] -> inconsistent
#   [1, 2, 3]  -> consistent


def is_consistent_literals(lits: Iterable[Literal]) -> bool:
    """Reject x and -x appearing together."""
    s = set(lits)
    return all(-x not in s for x in s)


# In DNF, term t1 subsumes term t2 if all literals of t1 are already in t2.
# Then t2 is more specific and can be removed as redundant.
```

```python
def subsumes_term(t1: Sequence[Literal], t2: Sequence[Literal]) -> bool:
    """t1 subsumes t2 in DNF iff literals(t1)  literals(t2)."""
    return set(t1).issubset(set(t2))


# In CNF, clause c1 subsumes clause c2 if all literals of c1 are already in c2.
# Then c2 is weaker or longer and can be removed as redundant.
def subsumes_clause(c1: Sequence[Literal], c2: Sequence[Literal]) -> bool:
    """c1 subsumes c2 in CNF iff literals(c1)  literals(c2)."""
    return set(c1).issubset(set(c2))


# This function removes redundant DNF terms.
# It does three main things:
#   1. remove inconsistent terms
#   2. remove duplicate terms
#   3. remove subsumed terms
# Finally, it sorts the result for stable output.
def remove_redundant_terms(dnf: DNF) -> DNF:
    """Remove duplicate/inconsistent/subsumed DNF terms."""
    canon = [list(canon_lits(t)) for t in dnf if is_consistent_literals(t)]
    out: DNF = []
    for t in canon:
        if any(subsumes_term(u, t) for u in canon if u != t):
            continue
        if t not in out:
            out.append(t)
    return sorted(out, key=lambda t: (len(t), [abs(x) for x in t], t))


# This function removes redundant CNF clauses.
# It does three main things:
#   1. remove inconsistent clauses
#   2. remove duplicate clauses
#   3. remove subsumed clauses
# Finally, it sorts the result for stable output.
def remove_redundant_clauses(cnf: CNF) -> CNF:
    """Remove duplicate/inconsistent/subsumed CNF clauses."""
    canon = [list(canon_lits(c)) for c in cnf if is_consistent_literals(c)]
    out: CNF = []
    for c in canon:
        if any(subsumes_clause(d, c) for d in canon if d != c):
            continue
        if c not in out:
            out.append(c)
    return sorted(out, key=lambda c: (len(c), [abs(x) for x in c], c))


# ============================================================
# SymPy conversion helpers
# ============================================================

# This function creates SymPy variables:
# x1, x2, x3, ..., xn
# It returns them in a dictionary so we can access them by number.
# Example:
#   var_symbols(3) gives
#   {1: x1, 2: x2, 3: x3}


def var_symbols(n: int) -> Dict[int, Symbol]:
    return {i: Symbol(f"x{i}") for i in range(1, n + 1)}


# This function converts one integer literal into a SymPy literal.
# Example:
#   3  -> x3
#  -3  -> Not(x3)
# The dictionary sym tells us which SymPy symbol belongs to each variable number.
def lit_to_sympy(lit: Literal, sym: Dict[int, Symbol]):
    v = sym[abs(lit)]
```

```
    return v if lit > 0 else Not(v)


# This function converts a DNF in list form into a SymPy Boolean expression.
# Example:
#   [[1,2],[-1,3]]
# becomes
#   (x1 & x2) | (~x1 & x3)
#
# Important special cases:
# - empty DNF [] means False
# - empty term [] means True
def dnf_to_expr(dnf: DNF):
    if not dnf:
        return false
    n = highest_var(dnf)
    sym = var_symbols(max(1, n))
    terms = []
    for term in dnf:
        if not term:
            return true
        terms.append(And(*[lit_to_sympy(l, sym) for l in canon_lits(term)]))
    return Or(*terms)


# This function converts a CNF in list form into a SymPy Boolean expression.
# Example:
#   [[1,2],[-1,3]]
# becomes
#   (x1 | x2) & (~x1 | x3)
#
# Important special cases:
# - empty CNF [] means True
# - empty clause [] means False
def cnf_to_expr(cnf: CNF):
    if not cnf:
        return true
    n = highest_var(cnf)
    sym = var_symbols(max(1, n))
    clauses = []
    for cls in cnf:
        if not cls:
            return false
        clauses.append(Or(*[lit_to_sympy(l, sym) for l in canon_lits(cls)]))
    return And(*clauses)


# This helper returns expr.args when the SymPy object has arguments.
# If not, it returns an empty list.
# It is used so that the parser can work for both simple and compound expressions.
def _expr_args(expr):
    return list(expr.args) if hasattr(expr, "args") else []


# This helper converts a SymPy atomic variable back into an integer.
# Example:
#   x3 -> 3
# It reads the variable name as text.
def _sympy_atom_to_int(p) -> int:
    s = str(p)
    if s.startswith('x'):
        return int(s[1:])
    return int(s)


# This function converts a SymPy Boolean expression into our CNF list format.
# It first asks SymPy to put the formula into CNF.
# Then it reads each clause and each literal and converts them back to integers.
#
# Special cases:
# - true  -> []
```

```
# - false -> [[]]
def expr_to_cnf_clauses(expr) -> CNF:
    """Convert a SymPy expression into our CNF list format."""
    expr = to_cnf(expr, simplify=True, force=True)
    if expr == true:
        return []
    if expr == false:
        return [[]]

    # This inner function reads one clause from a SymPy expression.
    # If the clause is an OR, we read all parts.
    # If it is a single literal, we still treat it as one clause.
    # Negative literals are converted back to negative integers.
    def parse_clause(e) -> Clause:
        parts = _expr_args(e) if e.func == Or else [e]
        lits: List[Literal] = []
        for p in parts:
            if p.func == Not:
                lits.append(-_sympy_atom_to_int(p.args[0]))
            else:
                lits.append(_sympy_atom_to_int(p))
        return list(canon_lits(lits))

    # If the full expression is an AND, each argument is one clause.
    # Otherwise the full expression itself is one clause.
    clauses = [parse_clause(a) for a in (
        _expr_args(expr) if expr.func == And else [expr])]

    # We clean the result by removing duplicate or subsumed clauses.
    return remove_redundant_clauses(clauses)


# This function converts a SymPy Boolean expression into our DNF list format.
# It first asks SymPy to put the formula into DNF.
# Then it reads each term and each literal and converts them back to integers.
#
# Special cases:
# - false -> []
# - true  -> [[]]
def expr_to_dnf_terms(expr) -> DNF:
    """Convert a SymPy expression into our DNF list format."""
    expr = to_dnf(expr, simplify=True, force=True)
    if expr == false:
        return []
    if expr == true:
        return [[]]

    # This inner function reads one term from a SymPy expression.
    # If the term is an AND, we read all parts.
    # If it is a single literal, we still treat it as one term.
    # Negative literals are converted back to negative integers.
    def parse_term(e) -> Term:
        parts = _expr_args(e) if e.func == And else [e]
        lits: List[Literal] = []
        for p in parts:
            if p.func == Not:
                lits.append(-_sympy_atom_to_int(p.args[0]))
            else:
                lits.append(_sympy_atom_to_int(p))
        return list(canon_lits(lits))

    # If the full expression is an OR, each argument is one term.
    # Otherwise the full expression itself is one term.
    terms = [parse_term(a) for a in (
        _expr_args(expr) if expr.func == Or else [expr])]

    # We clean the result by removing duplicate or subsumed terms.
    return remove_redundant_terms(terms)


# This is a direct helper for:
```

```python
#   DNF -> SymPy expression -> CNF list
# It is used in the main solver pipeline.
def dnf2cnf(dnf: DNF) -> CNF:
    return expr_to_cnf_clauses(dnf_to_expr(dnf))


# This is a direct helper for:
#   CNF -> SymPy expression -> DNF list
# It is also used in the main solver pipeline.
def cnf2dnf(cnf: CNF) -> DNF:
    return expr_to_dnf_terms(cnf_to_expr(cnf))


# This function tests whether two DNFs are logically equivalent.
# It builds the XOR of the two expressions and checks satisfiability.
# If XOR is unsatisfiable, then the two formulas are equivalent.
def equivalent_dnf(d1: DNF, d2: DNF) -> bool:
    """Logical equivalence test using satisfiability of XOR."""
    return satisfiable(dnf_to_expr(d1) ^ dnf_to_expr(d2)) is False


# ============================================================
# Resolution closure
# ============================================================

# This function checks whether a clause is a tautology.
# A clause is a tautology if it contains both x and -x.
# Example:
#   [1, -1, 3] is a tautology
# Such a clause is always true, so it is usually ignored in resolution.
def is_tautology_clause(cls: Sequence[Literal]) -> bool:
    s = set(cls)
    return any(-x in s for x in s)


# This function tries to compute one resolvent from two clauses.
# Resolution works only when there is exactly one pivot:
#   x in one clause, and -x in the other
#
# If there are zero pivots or more than one pivot, the function returns None.
# If the resolvent becomes a tautology, it also returns None.
def resolvent(c1: Sequence[Literal], c2: Sequence[Literal]) -> Optional[Clause]:
    """
    Compute a resolvent if c1 and c2 have exactly one pivot x / -x.
    If there are zero or multiple pivots, return None.
    """
    s1, s2 = set(c1), set(c2)
    pivots = [x for x in s1 if -x in s2]
    if len(pivots) != 1:
        return None
    p = pivots[0]
    res = list((s1 - {p}) | (s2 - {-p}))
    if is_tautology_clause(res):
        return None
    return list(canon_lits(res))


# This function computes the full resolution closure of a CNF.
# In simple words:
# - start with the original CNF clauses
# - try all pairs of clauses
# - add new resolvents
# - remove clauses that are redundant by subsumption
# - repeat until no new clause appears
#
# The final result is the saturated set of clauses.
def resolution_closure(cnf: CNF) -> CNF:
    """
    Saturate the CNF under resolution.
    Also apply a light subsumption cleanup after adding new clauses.
    """
```

```
    # First clean the input CNF:
    # - remove duplicates
    # - remove inconsistent clauses
    # - remove tautological clauses
    clauses = {canon_lits(c) for c in remove_redundant_clauses(
        cnf) if not is_tautology_clause(c)}

    # The loop continues as long as we keep finding new clauses.
    changed = True
    while changed:
        changed = False
        current = list(clauses)
        new_items: Set[Tuple[Literal, ...]] = set()

        # Try every pair of current clauses.
        for i in range(len(current)):
            for j in range(i + 1, len(current)):
                r = resolvent(list(current[i]), list(current[j]))
                if r is None:
                    continue
                rt = canon_lits(r)

                # If an old clause already subsumes the new one,
                # then the new one is not useful, so skip it.
                if any(set(c).issubset(set(rt)) for c in clauses):
                    continue

                new_items.add(rt)

        # If we found any new clauses, add them and clean again.
        if new_items:
            clauses |= new_items

            # Remove clauses that are subsumed by smaller clauses.
            clauses = {c for c in clauses if not any(
                (set(d).issubset(set(c)) and d != c) for d in clauses)}

            changed = True

    # Return the final closure in a stable sorted list form.
    return [list(c) for c in sorted(clauses, key=lambda c: (len(c), [abs(x) for x in c], c))]


# ============================================================
# Consensus closure for DNF
# ============================================================

# This section works on DNF formulas.
# The goal here is to compute consensus terms and then
# build the full consensus closure of a DNF.
#
# In simple words:
# - take two terms
# - if they differ in exactly one opposite literal
# - combine them into a new smaller term
# - keep doing this until no new term can be added

# This function tries to compute the consensus of two DNF terms.
# It works only when:
# - the two terms have exactly one complementary pair, like x and -x
# - and the rest of the literals do not create a contradiction
#
# If that condition is not satisfied, the function returns None.


def consensus_term(t1: Sequence[Literal], t2: Sequence[Literal]) -> Optional[Term]:
    """
    Compute the consensus of two DNF terms if they differ in exactly
    one complementary literal x / -x and are otherwise compatible.
    """
    # Convert both terms into sets for easier comparison.
```

```
    s1, s2 = set(t1), set(t2)

    # Find literals x in the first term such that -x is in the second term.
    pivots = [x for x in s1 if -x in s2]

    # Consensus is defined only when there is exactly one pivot.
    # If there are zero or more than one, return None.
    if len(pivots) != 1:
        return None

    # Take the single pivot.
    p = pivots[0]

    # Merge both terms, but remove the pivot pair p and -p.
    merged = (s1 | s2) - {p, -p}

    # If the merged term contains both x and -x for some variable,
    # then it is inconsistent, so return None.
    if not is_consistent_literals(merged):
        return None

    # Return the new consensus term in canonical order.
    return list(canon_lits(merged))

# This function computes the full consensus closure of a DNF.
# In simple words:
# - start with the original terms
# - try all pairs
# - add any new consensus terms
# - repeat until nothing new appears


def consensus_closure(dnf: DNF) -> DNF:
    """
    Repeatedly add all possible consensus terms until a fixed point.
    """
    # Start from the cleaned version of the input DNF.
    # Terms are stored as canonical tuples inside a set,
    # so duplicates are automatically removed.
    closure = {canon_lits(t) for t in remove_redundant_terms(dnf)}

    # Keep looping as long as new terms are found.
    changed = True
    while changed:
        changed = False
        terms = list(closure)

        # Try every pair of terms in the current closure.
        for i in range(len(terms)):
            for j in range(i + 1, len(terms)):
                c = consensus_term(terms[i], terms[j])

                # If no valid consensus exists, skip.
                if c is None:
                    continue

                ct = canon_lits(c)

                # Add the new consensus term only if it is not already present.
                if ct not in closure:
                    closure.add(ct)
                    changed = True

        # If we added anything new, clean the closure again.
        # This removes redundant terms and keeps the set minimal.
        if changed:
            closure = {canon_lits(t) for t in remove_redundant_terms(
                [list(t) for t in closure])}

    # Return the final closure as a sorted list of terms.
    return [list(t) for t in sorted(closure, key=lambda t: (len(t), [abs(x) for x in t], t))]
```

```python
# ============================================================
#  solver
# ============================================================

# This dataclass stores the full result of one run.
# It keeps all important intermediate objects, so they can be:
# - printed in the terminal
# - converted to JSON
# - inspected for debugging
@dataclass
class SolverResult:
    # Original input DNF after cleanup
    input_dnf: DNF

    # CNF version of the input
    phi_c: CNF

    # Resolution closure of phi_c
    closure: CNF

    # All-negative clauses from the closure
    phi_N: CNF

    # All other clauses from the closure
    phi_OP: CNF

    # psi2 = negation of phi_N, written as DNF terms
    psi2: DNF

    # psi1 = DNF version of phi_OP
    psi1: DNF

    # Whether psi1 is already a Horn-DNF
    psi1_is_horn: bool

    # Consensus closure of psi1  psi2
    closure_dnf: DNF

    # Only the Horn terms from the consensus closure
    horn_part: DNF

    # The non-Horn terms that remain in the consensus closure
    non_horn_terms: DNF

    # Whether the full closure_dnf is equivalent to the Horn-only part
    horn_equivalent_after_consensus: bool

    # Final success/fail answer for the run
    success: bool

    # Final phi1 if success holds
    phi1: Optional[DNF]

    # Final phi2 if success holds
    phi2: Optional[DNF]

    # Convert the dataclass into a dictionary.
    # This is useful for JSON output.
    def to_dict(self):
        return asdict(self)


# This is the main solver for the run.
# It follows the pipeline described in your report:
# 1. clean input
# 2. DNF -> CNF
# 3. resolution closure
# 4. split into phi_N and phi_OP
# 5. build psi2 and psi1
```

```
# 6. compute consensus closure
# 7. keep Horn terms only
# 8. test whether Horn-only part is equivalent
def solve_formula(dnf: DNF) -> SolverResult:
    """
    Execute the full solver pipeline.
    """
    # First remove redundant or inconsistent terms from the input.
    clean_input = remove_redundant_terms(dnf)

    # Step 1: Convert the cleaned DNF into CNF.
    phi_c = dnf2cnf(clean_input)

    # Step 2: Compute the resolution closure of the CNF.
    closure = resolution_closure(phi_c)

    # Step 3: Split the closure into:
    # - phi_N  = all-negative clauses
    # - phi_OP = all other clauses
    phi_N = [c for c in closure if all_negative(c)]
    phi_OP = [c for c in closure if not all_negative(c)]

    # Step 4: Build psi2 by negating each clause in phi_N.
    # Since phi_N contains all-negative clauses,
    # psi2 should become an all-positive DNF.
    psi2 = remove_redundant_terms([negate_clause(c) for c in phi_N])

    # Step 5: Convert phi_OP into DNF.
    # This gives psi1, which is the candidate Horn-DNF side.
    psi1 = cnf2dnf(phi_OP)

    # Check whether psi1 is already Horn-DNF.
    # This means every term must have at most one negative literal.
    psi1_is_horn = all(is_horn_term(t) for t in psi1)

    # Step 6: Compute the consensus closure of psi1  psi2.
    # In the code, union of DNF terms is represented by list concatenation.
    closure_dnf = consensus_closure(psi1 + psi2)

    # Step 7: Keep only the Horn terms from the closure.
    horn_part = remove_redundant_terms(
        [t for t in closure_dnf if is_horn_term(t)])

    # Also record the non-Horn terms that remain.
    non_horn_terms = remove_redundant_terms(
        [t for t in closure_dnf if not is_horn_term(t)])

    # Test whether the full consensus closure is equivalent
    # to the Horn-only part.
    # If yes, then all non-Horn terms are redundant.
    horn_equivalent_after_consensus = equivalent_dnf(closure_dnf, horn_part)

    # Final success means the Horn-only part is equivalent
    # to the whole closure.
    success = horn_equivalent_after_consensus

    # If successful, phi1 is the Horn-only part and phi2 is psi2.
    # If not successful, leave them as None.
    phi1 = horn_part if success else None
    phi2 = psi2 if success else None

    # Return all results together in one structured object.
    return SolverResult(
        input_dnf=clean_input,
        phi_c=phi_c,
        closure=closure,
        phi_N=phi_N,
        phi_OP=phi_OP,
        psi2=psi2,
        psi1=psi1,
        psi1_is_horn=psi1_is_horn,
```

```
        closure_dnf=closure_dnf,
        horn_part=horn_part,
        non_horn_terms=non_horn_terms,
        horn_equivalent_after_consensus=horn_equivalent_after_consensus,
        success=success,
        phi1=phi1,
        phi2=phi2,
    )


# ============================================================
# Input parsing
# ============================================================

# This function reads a DNF written as text and converts it into
# the list format used by the program.
#
# Example valid input:
#   [[1, 2, -3], [-1, 4]]
#
# Rules:
# - the whole input must be a list
# - each term must also be a list
# - each literal must be a nonzero integer
def parse_dnf(text: str) -> DNF:
    """
    Parse DNF entered as a Python-style list, for example:

        [[1, 2, -3], [-1, 4]]

    Each inner list is a term.
    """
    # Try to safely parse the text into a Python object.
    try:
        obj = ast.literal_eval(text)
    except Exception as e:
        raise ValueError(f"Could not parse DNF: {e}") from e

    # The whole expression must be a list.
    if not isinstance(obj, list):
        raise ValueError("DNF must be a list of terms.")

    parsed: DNF = []

    # Each item in the outer list should be one term.
    for term in obj:
        # Each term must itself be a list.
        if not isinstance(term, list):
            raise ValueError("Each term must be a list of integers.")

        # Each literal must be an integer and cannot be zero.
        if not all(isinstance(x, int) and x != 0 for x in term):
            raise ValueError("Each literal must be a nonzero integer.")

        parsed.append(term)

    # Return the validated DNF.
    return parsed


# This function asks the user to enter a DNF manually in the terminal.
# It prints an example to show the expected format.
# Then it reads one line of input and sends it to parse_dnf.
def interactive_input() -> DNF:
    """
    Ask the user for a DNF interactively.
    """
    banner("INTERACTIVE MODE")
    print("Enter a DNF as a Python-style list of terms.")
    print("Example: [[1, 2, 3, 4, -5], [1, 5, -4, -3, -2], [2, 4, 5, -1]]")
    print()
```

```
    raw = input("DNF> ").strip()
    return parse_dnf(raw)


# This function prints the final result in a readable terminal format.
# It shows:
# - summary
# - input
# - counts
# - candidates psi1 and psi2
# - final decision
# - optional full details when full=True
def pretty_print_result(result: SolverResult, full: bool = False) -> None:
    banner("SOLVER")

    # Print a one-line summary first.
    print(f"{C.DIM}Summary: {len(result.input_dnf)} input terms  {len(result.phi_c)} CNF clauses  {'SUCCESS'
     if result.success else 'FAIL'}{C.RESET}")

    # Input section
    section("Input")
    show_line("Input DNF", result.input_dnf)

    # Counts section
    # These counts help the user understand how the formula changes
    # through the pipeline.
    section("Counts")
    show_count("phi_c clauses", len(result.phi_c))
    show_count("closure clauses", len(result.closure))
    show_count("phi_N clauses", len(result.phi_N))
    show_count("phi_OP clauses", len(result.phi_OP))
    show_count("psi2 terms", len(result.psi2))
    show_count("psi1 terms", len(result.psi1))
    show_count("consensus terms", len(result.closure_dnf))
    show_count("non-Horn terms", len(result.non_horn_terms))

    # Candidate formulas section
    section("Candidates")
    show_line("psi2 = phi_N", result.psi2)
    show_line("psi1 = DNF(phi_OP)", result.psi1)
    print(f"{C.BOLD}{'psi1 Horn-DNF?':<28}{C.RESET} {status_word(result.psi1_is_horn)}")
    # print(f"{C.BOLD}{'Horn-equivalent?':<28}{C.RESET}
     {status_word(result.horn_equivalent_after_consensus)}")

    # Final result section
    section("Final result")
    print(f"{C.BOLD}{'RESULT':<28}{C.RESET} {status_word(result.success)}")

    # If success, print phi1 and phi2.
    if result.success:
        show_line("phi1", result.phi1, max_len=140)
        show_line("phi2", result.phi2, max_len=140)
        print(
            f"{C.GREEN}Result: SUCCESS  obtained phi1 and phi2 for the run.{C.RESET}")

    # If fail, print the blocking non-Horn terms.
    else:
        show_line("Blocking non-Horn terms", result.non_horn_terms)
        print(
            f"{C.YELLOW}Result: FAIL  essential non-Horn term remains after consensus.{C.RESET}")

    # If full=True, print more detailed intermediate formulas.
    if full:
        section("Full details")
        show_line("CNF phi_c", result.phi_c, max_len=500)
        show_line("Resolution closure", result.closure, max_len=500)
        show_line("phi_N", result.phi_N, max_len=500)
        show_line("phi_OP", result.phi_OP, max_len=500)
        show_line("Consensus closure", result.closure_dnf, max_len=500)
        show_line("Horn-only part", result.horn_part, max_len=500)
```

```
# ============================================================
# CLI entry point
# ============================================================
# These are the default examples built into the program.
# They are small examples used for quick testing.
#
# Example 1:
#   [[1, -2, -3], [4]]
#   This is DNF, but not Horn-DNF,
#   because the term [1, -2, -3] has two negative literals.
#
# Example 2:
#   [[1, 2], [-1, 3], [4]]
#   This is Horn-DNF,
#   because each term has at most one negative literal.
DEFAULT_EXAMPLES = [
    [[1, -2, -3], [4]],  # DNF, not Horn-DNF
    [[1, 2], [-1, 3], [4]],  # Horn-DNF
]


# This function builds the command-line parser.
# It defines all terminal options the program accepts.
#
# Supported options:
# - --dnf         : provide a DNF directly as text
# - --interactive : enter a DNF manually in the terminal
# - --example     : run one built-in example
# - --json        : print JSON output
# - --full        : show more detailed output
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Test a DNF formula for Horn-DNF and all-positive DNF decomposition."
    )

    # This option lets the user pass a DNF directly on the command line.
    # Example:
    #   --dnf "[[1,2,-3],[-1,4]]"
    parser.add_argument(
        "--dnf",
        type=str,
        help='DNF as a Python-style list, e.g. --dnf "[[1,2,-3],[-1,4]]"',
    )

    # This option puts the program into interactive mode.
    # The user is asked to type a DNF in the terminal.
    parser.add_argument(
        "--interactive",
        action="store_true",
        help="Enter the DNF interactively from the terminal.",
    )

    # This option runs one of the built-in examples.
    # Only 1 or 2 are allowed here because there are currently two defaults.
    parser.add_argument(
        "--example",
        type=int,
        choices=[1, 2],
        help="Run one of the built-in examples (1 or 2).",
    )

    # This option prints the final result in JSON format.
    # This is useful for debugging or saving machine-readable output.
    parser.add_argument(
        "--json",
        action="store_true",
        help="Print the full result object as JSON instead of fancy terminal output.",
    )

    # This option prints additional internal details.
```

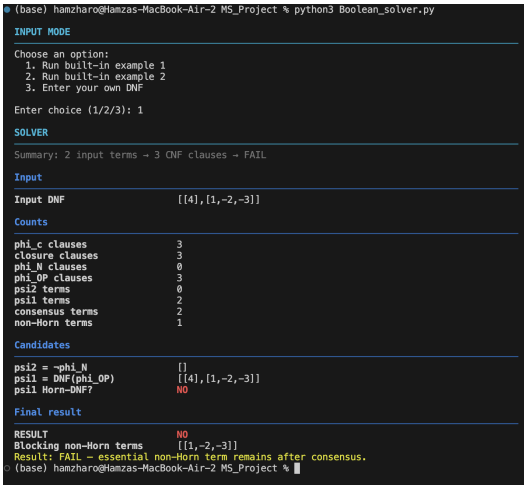
```
    # Without --full, the terminal output stays compact.
    parser.add_argument(
        "--full",
        action="store_true",
        help="Show full formulas instead of compact summary output.",
    )

    return parser


# This is the main entry point of the program.
# It decides how the input DNF will be obtained:
# - from --dnf
# - from interactive input
# - from a built-in example
# - or from the small menu when no arguments are given
def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    # Case 1:
    # User gave a DNF directly with --dnf
    if args.dnf:
        dnf = parse_dnf(args.dnf)
        result = solve_formula(dnf)

    # Case 2:
    # User asked for interactive input
    elif args.interactive:
        dnf = interactive_input()
        result = solve_formula(dnf)

    # Case 3:
    # User asked to run one built-in example
    elif args.example:
        dnf = DEFAULT_EXAMPLES[args.example - 1]
        result = solve_formula(dnf)

    # Case 4:
    # No command-line input was given,
    # so show a small terminal menu
    else:
        banner("INPUT MODE")
        print("Choose an option:")
        print("  1. Run built-in example 1")
        print("  2. Run built-in example 2")
        print("  3. Enter your own DNF")
        print()

        choice = input("Enter choice (1/2/3): ").strip()

        # Built-in example 1
        if choice == "1":
            dnf = DEFAULT_EXAMPLES[0]

        # Built-in example 2
        elif choice == "2":
            dnf = DEFAULT_EXAMPLES[1]

        # Manual user input
        elif choice == "3":
            dnf = interactive_input()

        # Invalid choice
        else:
            print("Invalid choice.")
            return

        # Solve the chosen input
        result = solve_formula(dnf)
```

```
    # If the user asked for JSON output,
    # print the whole result object as JSON
    if args.json:
        print(json.dumps(result.to_dict(), indent=2))

    # Otherwise print the formatted terminal view
    else:
        pretty_print_result(result, full=args.full)


# This makes the file executable as a script.
# When the user runs:
#   python Boolean_solver.py
# Python enters here and starts main().
if __name__ == "__main__":
    main()


# This list contains more test examples.
# These are not used directly by main(),
# but they are useful for future experiments, debugging, or extension.
#
# Notes:
# - some are Horn-DNF
# - some are not Horn-DNF
# - some are mixed examples
# - these examples can help test different behaviors of the solver
TEST_EXAMPLES = [
    [[1, -2, -3], [4]],            # DNF, not Horn-DNF
    [[1, 2], [-1, 3], [4]],        # Horn-DNF
    [[1], [-2, 3]],                # Horn-DNF
    [[1, -2, -3]],                 # DNF, not Horn-DNF
    [[1, -2], [3], [4, 5]],        # Horn-DNF
    [[1, -2], [2, -3], [3, -4]],   # mixed DNF, interesting test
    [[-1, 2], [1, -3], [3, 4]],    # mixed DNF, interesting test
    [[1, -2, -3, 5], [4], [-1, 3]]  # mixed DNF, likely non-Horn
]
```

Appendix B: Example Runs

This appendix presents two sample executions of the solver. The first example is a failure case, while the second example is a success case. Each screenshot is shown together with a short explanation of the output.

Example 1: Failure Case



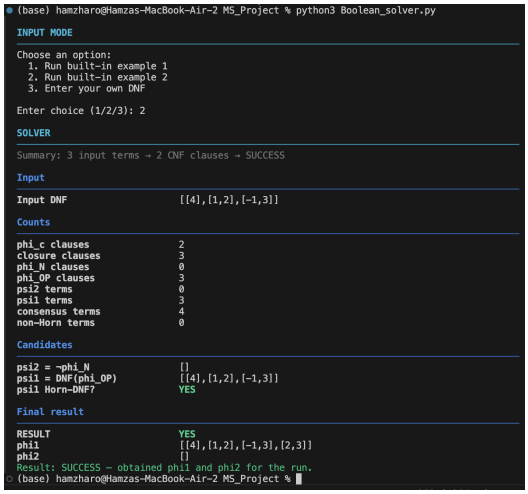
Input:

$$[[4],[1,-2,-3]]$$

Explanation:

- The summary indicates that the input contains 2 terms, which are converted into 3 CNF clauses.
- The final result is **FAIL**.
- The line `psi1 Horn-DNF? NO` shows that ψ_1 is not a Horn-DNF.
- This occurs because the term $[1,-2,-3]$ contains multiple negative literals.
- The solver reports the blocking non-Horn term $[1,-2,-3]$.
- After the consensus step, a non-Horn term still remains essential in the formula.
- Consequently, this formula does not satisfy the required condition for this method.

Example 2: Success Case



Input:

$$[[4],[1,2],[-1,3]]$$

Explanation:

- The summary indicates that the input contains 3 terms, which are converted into 2 CNF clauses.
- The final result is **SUCCESS**.
- The line `psi1 Horn-DNF? YES` shows that ψ_1 already satisfies the Horn-DNF condition.
- Every term in ψ_1 has at most one negative literal.
- The line `YES` confirms that the formula satisfies the required condition.
- The solver returns ϕ_1 and ϕ_2.
- Here, ϕ_1 is the Horn-DNF part, and $\phi_2 = []$, meaning there is no negative-clause contribution in this example.

Figure 2: Two example runs of the solver.